

Evaluating Pull Requests: An Early Warning Model for Global Open-Source vs. Academic Projects

Cao-Cuong Dang

Department of Information Technology
FPT University
Hanoi, Vietnam
caocuong7a2@gmail.com

The-Viet Dao

Department of Information Technology
FPT University
Hanoi, Vietnam
daothevietnb123@gmail.com

Duc-Thinh Tran

Department of Information Technology
FPT University
Hanoi, Vietnam
thinhtranktyt11@gmail.com

Van-Hieu Dang

Department of Information Technology
FPT University
Hanoi, Vietnam
hieudv@fpt.edu.vn

Abstract—Modern software development relies heavily on the Pull Request (PR) workflow for code integration and quality assurance. However, code review practices and PR characteristics vary significantly across different development environments. This paper presents a comparative analysis of PR review cultures between global open-source software (OSS) projects and academic student projects at FPT University. By analyzing a dataset of 1,771 cleaned PRs, we highlight key behavioral differences in review time, code change sizes, review comments, and rejection rates. Based on these insights, we develop and evaluate three predictive models as an Early Warning System to flag high-risk PRs before merge decisions: a Gaussian Anomaly Detection model, a Random Forest Classifier, and an Average Voting Ensemble model. Our findings show that the Random Forest Classifier achieves 95.00% Precision (Recall of 51.35%) in identifying unsuccessful PRs, while the Gaussian Anomaly Detection model provides 100.00% Recall. The Ensemble model balances the trade-offs with 86.49% Recall and 13.85% Precision. We discuss critical factors influencing PR outcomes, analyze model feature importances, and address the data leakage limitation in pre-merge prediction models.

Index Terms—Pull Request, Code Review, Early Warning System, Software Quality, Anomaly Detection, Random Forest, Ensemble Learning, Academic Projects.

I. INTRODUCTION

Modern software engineering is highly collaborative, with platforms like GitHub serving as the standard infrastructure for version control and peer code review. In this ecosystem, the Pull Request (PR) workflow is the cornerstone of quality assurance, allowing developers to propose code changes that are subsequently reviewed, tested, and integrated. Despite its widespread adoption, the execution of the PR workflow is deeply influenced by organizational culture, developers' experience, and project constraints.

Specifically, there exists a stark contrast between professional global open-source software (OSS) communities and academic environments (such as university courses). Global OSS projects are characterized by distributed, asynchronous communication, rigorous continuous integration (CI) tests, and

strict review standards. In contrast, academic projects—run by students under tight semester deadlines—often exhibit different behavior due to limited experience and the pressure of immediate course evaluations.

Understanding these differences is crucial for two main reasons. First, it helps educators identify common anti-patterns in students' Git workflows, enabling them to align academic training with industry standards. Second, it allows the creation of automated assistance systems to optimize the code review process.

This study conducts a comparative analysis of Git PRs from global repositories and student repositories at FPT University. Additionally, we propose an Early Warning System utilizing a multivariate Gaussian Anomaly Detection model, a supervised Random Forest Classifier, and an Average Voting Ensemble model to predict PR rejection risks based on pre-merge metrics, assisting reviewers in prioritizing high-risk changes.

The rest of this paper is organized as follows: Section II discusses the related work. Section III outlines our data collection methodology and statistical comparative findings. Section IV describes the development and evaluation of our Early Warning Models. Section V discusses the results, model trade-offs, and critical data leakage limitations. Section VI summarizes our findings and proposes future research directions.

II. RELATED WORK

Collaborative software development relies extensively on peer code reviews, a practice pioneered and popularized in open-source software (OSS) communities and subsequently adopted across the industry. Extensive research has examined code review workflows on platforms like GitHub. Studies show that code reviews not only improve code quality but also foster knowledge sharing within development teams.

However, the majority of literature focuses on professional settings or large-scale global OSS projects. Academic software development practices—specifically those in university

courses—remain understudied. In university environments, students often face constraints such as immediate deadlines and lack of professional experience, which lead to distinct development behaviors. Understanding the gap between academic code review practices and industry standards is key to improving computer science pedagogy.

Furthermore, automated tools to evaluate Pull Request (PR) quality and predict integration risks are gaining traction. Predictive models have been built to estimate review time or the likelihood of a PR being merged. However, many of these models utilize complex machine learning pipelines that are difficult to interpret. In this paper, we propose and compare highly interpretable models, including Gaussian Anomaly Detection and Random Forest, to serve as an early warning system, using basic pre-merge characteristics of PRs to predict rejection risks, which is suitable for both student training and developer assistance.

III. METHODOLOGY AND COMPARATIVE ANALYSIS

In this section, we describe our data collection methodology, database storage schema, data cleaning pipeline, and present a comparative analysis of PR characteristics between global OSS projects and academic student projects at FPT University.

A. Data Collection and Storage Schema

We developed Python scripts utilizing the GitHub REST API to collect PR data. The raw dataset consists of two groups:

- **Global OSS Dataset:** 1,000 randomly sampled PRs from popular open-source repositories on GitHub.
- **FPT Academic Dataset:** 1,000 randomly sampled PRs from student repositories at FPT University, primarily from courses such as SWP391 (Software Development Project) and Capstone Projects.

To store and query the collected data efficiently while avoiding data redundancy, we designed a relational SQLite database structure comprising two main entities: `repositories` and `pull_requests`, linked via a one-to-many relationship. The schema is shown below:

- `repositories`: Stores repo metadata, including `repo_name` (unique key), `repo_language`, `repo_stars`, and `repo_forks`.
- `pull_requests`: Stores PR metrics, including `pr_id` (primary key), `repo_id` (foreign key), `pr_number`, `title`, `body_len`, `user_login`, `user_type`, `created_at`, `closed_at`, `merged_at`, `duration_minutes`, `comments`, `review_comments`, `commits`, `additions`, `deletions`, `changed_files`, `is_fpt`, and search query keyword.

We implemented a database view named `view_pr_clean` to standardize datetime formats, compute duration in hours, and define the binary target variable `is_merged` (1 if `merged_at` is not null, indicating a successful merge, and 0 if null, indicating a rejected or closed PR).

B. Data Preprocessing and Outliers Removal

A critical step in the data pipeline is removing extreme outliers. In software development, metrics like additions, deletions, commits, and changed files follow a heavily right-skewed power-law distribution. Large PRs (e.g., automated code changes or importing massive external libraries) do not represent typical human programming behavior and can bias machine learning models.

We established strict thresholds based on domain knowledge:

- `additions` $\leq 5,000$ lines of code
- `deletions` $\leq 5,000$ lines of code
- `changed_files` ≤ 50 files
- `commits` ≤ 50 commits

Applying these filters resulted in a cleaned dataset of **1,771 PRs** (929 Global OSS and 842 FPT Academic), removing 10.60% of the raw data. Interestingly, the outliers removal rate was much higher for the FPT student dataset (15.80% of FPT samples removed) compared to the Global dataset (only 5.30% of Global samples removed). This quantitatively highlights the lack of structured Git practices among students, who frequently commit massive, unstructured blocks of code rather than modular, incremental changes.

C. Comparative Analysis

We conducted statistical comparisons based on four primary aspects of the PR lifecycle:

- 1) **Pull Request Lifecycle:** The median lifetime (from creation to merge/close) of student PRs is significantly lower than that of global OSS projects. Under the pressure of short academic course deadlines, students exhibit a tendency to rush approvals, merging PRs quickly. Conversely, global OSS projects undergo extensive asynchronous reviews and automatic checks, leading to longer lifespans.
- 2) **Rejection and Revert Rates:** The rejection rate in the FPT student dataset is approximately 9.00%, which is higher than the 6.22% observed in the global OSS dataset. Detailed examination reveals that students often treat GitHub as a remote backup. When facing merge conflicts or uploading to the wrong branch, they tend to close the current PR and open a new one rather than resolving conflicts inline.
- 3) **Code Change Sizes:** Global OSS developers generally adhere to the "Atomic PR" principle, proposing small, focused changes. Conversely, student PRs are often large and disorganized, modifying many unrelated files simultaneously. This is due to the student habit of committing all weekly work in a single large PR.
- 4) **Review Culture:** We observed a lack of asynchronous code review comments on GitHub in the student repositories. The average comment count per PR in the student dataset is near zero. FPT students typically communicate via synchronous channels (face-to-face discussions or instant messaging apps) and approve PRs without logging

code-level feedback on GitHub. Global OSS projects, by contrast, exhibit rich discussion threads on GitHub due to geographical and time-zone differences.

IV. PROPOSED EARLY WARNING SYSTEM

Based on the comparative findings, we formulate and evaluate three models as an Early Warning System to predict the probability of a PR being rejected. Reviewers and educators can use this system to automatically flag high-risk PRs early.

A. Data Splitting and Harmonization

To evaluate the models on comparable grounds, we defined a harmonized data split: 70% training and 30% testing across the combined processed dataset. This split is stratified by the target variable `is_merged` to maintain class proportions:

- **Global Training Set:** 1,239 samples.
- **Global Test Set:** 532 samples.

The variables include numerical statistics (commits, comments, additions, deletions, changed files, duration) and one-hot encoded categorical variables for primary programming languages and keywords.

B. Model Architectures

1) *Gaussian Anomaly Detection:* Since PR rejections are relatively rare events (a minority class of 9.00% in FPT and 6.22% globally), we can frame PR failure prediction as an anomaly detection task. We train a multivariate Gaussian model strictly on "normal" (successful, merged) FPT samples in the training set (337 samples). The probability density $p(x)$ for a feature vector $x \in \mathbb{R}^d$ is modeled as:

$$p(x) = \prod_{j=1}^d \frac{1}{\sqrt{2\pi}\sigma_j} \exp\left(-\frac{(x_j - \mu_j)^2}{2\sigma_j^2}\right) \quad (1)$$

where μ_j and σ_j^2 are the mean and variance of the j -th feature. A validation set containing both normal and anomaly FPT samples (273 samples) was used to select the optimal threshold parameter $\epsilon = 1.4388 \times 10^{-8}$. If $p(x) < \epsilon$, the PR is flagged as an anomaly (Not Merged).

2) *Random Forest Classifier:* We train a supervised Random Forest Classifier on the global training set (1,239 samples). To handle the significant class imbalance between merged and unmerged PRs, we apply a balanced class weighting scheme. The model constructs 100 decision trees and aggregates their individual predictions to produce a final classification.

3) *Ensemble Model (Average Voting):* To combine the unsupervised anomaly detection capabilities of the Gaussian model with the supervised classification strength of the Random Forest, we implement an Average Voting Ensemble. For a given test sample, both models output binary predictions (0 for unmerged, 1 for merged). The ensemble computes the average prediction:

$$y_{ens} = \mathbb{I}\left(\frac{y_{gaus} + y_{rf}}{2} > 0.5\right) \quad (2)$$

where $\mathbb{I}$ is the indicator function. Because of the average threshold (> 0.5), this voting scheme behaves as a logical AND-gate for successful merges: a PR is classified as merged (1) only if both models predict 1. If either model flags it as unmerged (0), the ensemble flags it as unmerged.

C. Performance Evaluation

Table I presents the model evaluation results on the test set for the minority class of interest (Not Merged, class 0).

TABLE I
MODEL EVALUATION METRICS FOR THE "NOT MERGED" CLASS

Model	Precision	Recall	F1-Score
Gaussian Anomaly Detection	0.0829	1.0000	0.1532
Random Forest Classifier	0.9500	0.5135	0.6667
Ensemble (Average Voting)	0.1385	0.8649	0.2388

D. Feature Importances

We extracted the Gini feature importances from the trained Random Forest Classifier. Table II lists the top features contributing to the model's decisions.

TABLE II
RANDOM FOREST FEATURE IMPORTANCES

Feature	Importance
duration_minutes	0.1862
duration_hours	0.1853
additions	0.0799
deletions	0.0736
created_hour	0.0640
changed_files	0.0591
comments	0.0526
created_day_of_week	0.0342

V. DISCUSSION AND LIMITATIONS

A. Model Trade-offs and Voting Mechanics

The experimental results demonstrate clear trade-offs between the three modeling approaches. The Gaussian Anomaly Detection model achieved a perfect Recall of 100.00% but a low Precision of 8.29%. In practice, this means the model flagged almost all FPT PRs as anomalies. This behavior is due to the extreme variance and noise present in student repositories, where even successful merges deviate heavily from standard coding patterns, leading to low probability densities. Thus, the Gaussian model acts as a highly conservative screener.

The Random Forest Classifier, by contrast, achieved an exceptional Precision of 95.00% and a Recall of 51.35%. The model is highly selective: it only flags a PR as unmerged when there is strong, consistent evidence (such as huge additions or comments), minimizing false alarms but missing about half of the actual failures.

The Ensemble model represents a middle ground. Because of the voting logic, where an anomaly prediction (0) by either

model triggers an ensemble anomaly prediction, the ensemble inherits the high recall of the Gaussian model (86.49%) and slightly improves the precision to 13.85% (compared to 8.29% for the pure Gaussian model).

B. The Critical Limitation of Data Leakage

A key finding in our analysis of feature importances (Table II) is that `duration_minutes` and `duration_hours` are the most important predictors, accounting for over 37.00% of the model's split decisions.

However, this reveals a major methodological limitation: **Data Leakage**. The duration of a PR is a post-hoc metric calculated as the difference between `closed_at` and `created_at`. In a real-time warning system, when a developer first submits a PR, the duration is unknown. Training the model on duration features means we are feeding it information from the future. The high importance of duration arises because rejected PRs are often closed very quickly (if they are mistakes) or drag on indefinitely, making duration an easy proxy for the target variable `is_merged`.

To make this model practically viable as a true *early warning* tool, future iterations must omit post-merge duration metrics. The classifier should rely exclusively on pre-merge indicators available at PR creation time, such as code churn (additions, deletions), the number of affected files (`changed_files`), and historical repository characteristics.

VI. CONCLUSION AND FUTURE WORK

This paper presented a comparative study of Git Pull Request workflows between global open-source projects and academic student repositories. Our statistical analysis confirmed that student workflows suffer from lack of atomic commits, rushed merges under deadline pressure, and a lack of written code reviews. We evaluated three predictive models to serve as an early warning system. While the Random Forest Classifier demonstrated high precision (95.00%) and the Gaussian Anomaly Detection model provided maximum recall (100.00%), our feature analysis identified critical data leakage in the form of post-merge duration features.

In future work, we will re-train the models using exclusively pre-merge features to establish a realistic early warning system. Furthermore, we intend to implement this system as a lightweight GitHub Action that automatically scans student PRs upon creation, flagging code integration risks and fostering a healthier, industry-aligned Git review culture in academic settings.

REFERENCES

REFERENCES

- [1] G. Gousios, M. Pinzger, and A. van Deursen, "An exploratory study of the pull-based software development model," in Proc. Int. Conf. on Software Engineering (ICSE), 2014, pp. 345–355.
- [2] A. Bacchelli and C. Bird, "Expectations, outcomes, and challenges of modern code review," in Proc. Int. Conf. on Software Engineering (ICSE), 2013, pp. 712–721.
- [3] Y. Yu, M. Wang, and G. Yin, "Predicting reviewer response time in pull-based developer collaboration," in Proc. Conf. on Software Analysis, Evolution, and Reengineering (SANER), 2015, pp. 12–21.
- [4] F. Thung, T. F. Bissyande, D. Lo, and L. Jiang, "An empirical study of bugs in machine learning systems," in Proc. Int. Symposium on Software Reliability Engineering (ISSRE), 2012, pp. 273–282.
- [5] O. Kononenko, O. Baysal, and M. W. Godfrey, "Code review quality: How manual code review detects bugs," in Proc. Int. Conf. on Software Engineering (ICSE), 2016, pp. 119–130.
- [6] J. Tsay, L. Dabbish, and J. Herbsleb, "Influence of social and technical factors for evaluating contribution in GitHub," in Proc. Int. Conf. on Software Engineering (ICSE), 2014, pp. 356–366.